

Chapter 10

Planning an Iteration

Release planning has left us with a coarse-grained allocation of stories to the iterations that comprise a release. This level of planning—not so much detail that we give the false feeling of precision and accuracy, but enough planning that we can base actions on it— is ideal for planning a release. However, at the start of each iteration it is important to take the planning process one step further.

Iteration Planning Overview

To plan an iteration the whole team holds an iteration planning meeting. The customer as well as all of the developers (that is, programmers, testers and others) on the team attend and participate in this meeting. Because the team will be looking at the stories in detail, they will undoubtedly have some questions about them. They need the customer present to answer these questions.

The general sequence of activities for an iteration planning meeting is as follows:

1. Discuss a story.
2. Disaggregate the story into its constituent tasks.
3. One developer accepts responsibility for each task.
4. After all stories have been discussed and all tasks have been accepted, developers individually estimate the tasks they've accepted to make sure they are not over-committed.

Each of the activities is discussed in one of the following sections.

Discussing the Stories

As input to the iteration planning meeting, the team has the set of prioritized stories. Just as programmers may change their opinions about the difficulty of programming a story, the customer may change her mind about the priority of a story. The iteration planning meeting is the perfect time for the customer to express these priority changes to the team.

To start the meeting, the customer starts with her highest priority story and reads it to the developers. The developers then ask questions until they understand the story sufficiently to disaggregate it into constituent tasks. It is not necessary to understand every detail of the story, and diving too deeply into the details of each story can make the meeting very lengthy and inefficient, since not everyone in the meeting needs to hear all the detail on all stories. The developers will still be able to work out the fine details of the stories with the customer after the planning meeting.

Changing Priorities

It is usually best if the customer can withstand the desire to change priorities *during* an iteration. It is very easy for a team to get whipsawed during an iteration if the customer changes her mind frequently. On one project, for example, the customer and a programmer met and agreed about how a database search feature would work. Five days into a ten-day iteration (and about two-thirds through coding the search feature), the customer came up with what she thought was a better solution that was completely different from the original, partially coded solution. At that point, in her mind the customer was comparing two uncoded solutions and she naturally favored the one she thought was better. She urged the team to abandon the current approach and immediately start on the new approach. We politely asked her to wait until the end of the iteration and she agreed. At that point, she was comparing a fully working solution that did most of what she wanted in a search feature and another version that was undoubtedly better but that would take 10 days to develop.

Even though she (and the rest of the team) thought the new search feature would be better, it was not worth adding at that point when compared to a fully working adequate feature. Users were better served by having developers work on entirely new features.

Disaggregating into Tasks

There is really no art to disaggregating a story into tasks. Many developers have been doing this for much of their careers. Since stories are already fairly small (typically taking the project's archetypal programmer one to five days of ideal time) there is usually not that much disaggregation necessary.

In fact, why disaggregate at all? Why not leave the story alone as a discrete unit of work?

Even though stories are small enough to serve as units of work, projects are generally well served by disaggregating them into even smaller tasks. First, for many teams the story will not be implemented by just one developer (or one pair of developers). The story may be split among developers either because the developers specialize in certain technologies or because splitting the work is the fastest way to complete the story.

Second, stories are descriptions of user- or customer-valued functionality; they are not to-do lists for developers. The act of converting a story into its constituent tasks is often useful because it helps point out tasks that might have been forgotten. Because the disaggregation into tasks happens in a group setting, the entire strength of the team is brought to the effort. While one developer may forget that it's necessary to update the install program as part of a story, it is less likely that everyone will forget.

One of the criticisms of agile processes is that there is no upfront design step, as there is in a waterfall process. While it's true there is no upfront design *phase*, agile processes are characterized by frequent short bursts of design. Disaggregating stories into tasks—which can only be done with at least a minimal design in mind—is one of these short bursts of just-in-time design that replace a waterfall's upfront design phase.

As various team members call out the tasks that comprise a story, someone on the team needs to write the tasks on something. My personal preference is to write them on a white board in a shared team meeting room.

As an example of disaggregating a story into tasks, suppose we have the story “A user can search for a hotel on various fields.” That story might be turned into the following tasks:

- code basic search screen
- code advanced search screen
- code results screen
- write and tune SQL to query the database for basic searches

- write and tune SQL to query the database for advanced searches
- document new functionality in help system and user's guide

In particular notice the inclusion of the task for updating the user's guide and help system. Even though this story did not explicitly say anything about documentation, the team knew from prior iterations that there are a help system and a user's guide and that they need to be accurate at the end of each iteration. If there was any question about this the team could have asked the customer.

Guidelines

Because stories are already fairly small it is not necessary to set very precise guidelines around the desired size of a task. Use these guidelines when disaggregating stories into tasks:

- If one task of a story is particularly difficult to estimate (for example, a list of supported data formats requires approval from a remote Vice President who is slow to respond), separate that task from the rest of the story.
- If tasks could easily be done by separate developers, then split those tasks. For example, in the case above, coding of the basic and advanced search screens was separated. There may be some natural synergy to having the same programmer or pair work on both but it isn't necessary. Disaggregating tasks in this way is useful because it lets multiple developers work on the same story. This is frequently necessary near the end of an iteration as time starts to run out. Similarly, the task "code basic search screen" could have been split into two tasks: "Design screen layout for basic search screen" and "Code basic search screen" if the team is using a user interface designer or group.
- If there is benefit in knowing that a part of the story is done, break that part out as a task. In the example above, the coding of the basic and advanced search screens were made separate tasks. This will allow the developer who writes the database access code to connect her SQL to the search screens one at a time as they become available. This means that a delay in the completion of the advanced search screen does not delay completion of the two tasks for the basic search screen.

Accepting Responsibility

Once all the tasks for a story have been identified, someone on the team needs to volunteer to perform each task. If the tasks were written on a white board, developers simply write their names next to the tasks they are accepting.

Even if the team will be doing pair programming, it is generally best to associate a single name with each task. This person assumes responsibility for completing the task. If he needs additional information from the customer, he gets it. If he chooses to pair program, he solicits a pair. Ultimately, though, it is his responsibility to make sure the task gets completed during the iteration.

Actually, it's the responsibility of everyone on the team to make sure the task gets completed. The team needs a "we're all in this together" mentality. And, if near the end of the iteration, one developer is not going to complete all the tasks she accepted, then others on the team are expected to take on that work to the extent possible.

Even though an individual signs up for a task and accepts responsibility for it, these responsibilities must be fluid throughout the iteration. As the team progresses through the iteration, learning more about the tasks, finding some work easier than planned but some harder than planned, commitments need to change. At the end of an iteration no one can say "I finished my work, but Tom still had a few tasks left."

Estimate and Confirm

If a project team's velocity is forty story points per iteration, then the previous steps—discussing a story, disaggregating it into tasks, and accepting responsibility for the tasks—are repeated until the team has discussed the customer's top forty story points worth of stories. At that point each developer is responsible for estimating the amount of work she has accepted responsibility for. The best way to do this is still to estimate in ideal time.

By this point the tasks should be small enough that they can be estimated with some reliability. But, if not, don't worry about it. Take a best guess at the duration of the task and move on. If, as was recommended earlier in this chapter, the tasks and names of the responsible developers were written on a white

board, each developer can now add his or her estimates to the board. The result will look something like Table 10.1.

Table 10.1 *It's easy to track tasks, the developer doing each task, and estimates on a white board.*

Task	Who	Estimate
Code basic search screen	Susan	6
Code advanced search screen	Susan	8
Code results screen	Jay	6
Write and tune SQL to query the database for basic searches	Susan	4
Write and tune SQL to query the database for advanced searches	Susan	8
Document new functionality in help system and user's guide	Shannon	2

Once a developer has estimated each of her tasks, she needs to add them up and make a realistic assessment about whether they can all be completed during the iteration. For example, suppose a two-week iteration is beginning and I have accepted tasks that I now estimate will take 53 hours of actual time on task. It's doubtful that with everything else I have to do that I will be able to put that much time directly onto these tasks. At this point I have the following options:

- keep all the tasks and hope
- request that someone else on the team take some of my tasks
- talk with the customer about dropping a story (or splitting a story and dropping part of it)

If one of the developers finishes assessing her workload and feels she can take on my tasks, responsibility for them can move to her. If, however, no one has any additional capacity to take on the tasks, then the customer has to help by removing some work from the iteration. Each developer must feel comfortable committing to the work she's signed up for. And, since the team has a "we're all in this together" attitude, everyone needs to be comfortable with the overall commitment of the team.

Summary

- Iteration planning takes release planning one step further but only for the iteration being started.
- To plan the iteration, the team discusses each story and disaggregates it into its constituent tasks.
- There is no mandatory size range for tasks (for example, three to five hours). Instead, stories are disaggregated into tasks to facilitate estimation or to encourage more than one developer to work on various parts of the story.
- One developer accepts responsibility for each task.
- Developers assess whether they have over-committed themselves by estimating each task they have accepted.

Developer Responsibilities

- You are responsible for participating in the iteration planning meeting.
- You are responsible for helping to disaggregate all stories into tasks, not just the stories you are likely to work on.
- You are responsible for accepting responsibility for the tasks you will work on.
- You are responsible for ensuring you take on an appropriate amount of work.
- Throughout an iteration you are responsible for monitoring the amount of work you have left as well as the amount of work your teammates have left. If you're likely to finish your work, you are responsible for taking on some work from your teammates.

Customer Responsibilities

- You are responsible for prioritizing the stories that will be included in the iteration.
- You are responsible for directing the developers toward the greatest business value they can deliver. This means that if higher-value stories have come to light since the release plan was first established, you are responsible for adjusting priorities to deliver maximum business value.
- You are responsible for participating in the iteration planning meeting.

Questions

- 10.1 Disaggregate this story into its constituent tasks: A user can view detailed information about a hotel.